

---

# SEA-Rec: Semantic-Enhanced Aligned Generative Sequential Recommendation

Code repository: [https://github.com/Mancupfire/Data\\_Mining](https://github.com/Mancupfire/Data_Mining)

---

**Nguyen Tran Nhat Minh**  
VinUniversity, Hanoi, Vietnam  
24minh.ntn@vinuni.edu.vn

**Trinh To Thao Nhi**  
VinUniversity, Hanoi, Vietnam  
23nhi.ttt@vinuni.edu.vn

**Nguyen Hoang Nam**  
VinUniversity, Hanoi, Vietnam  
24nam.nh@vinuni.edu.vn

## Abstract

Sequential recommendation asks a model to predict a user’s next item from a short interaction history while ranking against a large catalog. We present **SEA-Rec**, a semantic-enhanced aligned generative sequential recommendation framework that represents each item as a short residual-quantized discrete code sequence and trains a T5-style encoder–decoder model to generate the next item code. The project focuses on tokenizer–recommender alignment: an RQ-VAE tokenizer learns codes that reconstruct item embeddings, whereas the recommender needs codes that are predictable from user histories. SEA-Rec investigates cyclic co-training, symmetric KL alignment, decoder-level contrastive learning, and a feature-adapter path. Experiments on three Amazon Review subsets show that final checkpoints consistently improve over their corresponding pre-test checkpoints, while SEA-Rec outperforms verified classical and sequential baselines on Scientific. Codebook analysis shows full utilization of all three 256-entry RQ-VAE levels with no dead codes.

## 1 Introduction

Consider a user with a short history such as [item A, item B, item C]. The recommender must predict the next item from tens of thousands of candidates. Under full-catalog evaluation, the held-out target is ranked against every catalog item, and a top- $K$  prediction is correct only when the exact target appears in the returned list.

Sparse interactions, short histories, and long-tailed item popularity make this setting difficult, especially on the Amazon Scientific subset. In this report, “semantic” refers to structure in the learned collaborative embedding and discrete code space, not to raw natural-language item metadata.

Generative recommendation offers an alternative to scoring every item independently. Instead of assigning each item only an atomic ID, SEA-Rec maps items to short discrete code sequences and trains a T5-style sequence-to-sequence model to generate the next item’s code. This lets related items share structure through their learned code paths.

The central challenge is alignment. The RQ-VAE tokenizer is naturally trained to reconstruct item embeddings, but the recommender needs item codes that are predictable from user histories. SEA-Rec therefore studies cyclic co-training, symmetric KL alignment, decoder-level contrastive learning, and a feature adapter that injects the original 256-dimensional item embedding into the encoder stream. We interpret the short-run ablations conservatively rather than claiming that every component is conclusively beneficial.

Our contributions are threefold: (1) an implementation of SEA-Rec with cyclic co-training between an RQ-VAE tokenizer and a T5-style generative recommender, (2) empirical evaluation on three Amazon Review subsets under full-catalog ranking, and (3) dataset and codebook analysis showing the effects of sparsity, short histories, long-tailed popularity, and residual code utilization.

## 2 Related Work

**Sequential recommendation.** Bayesian Personalized Ranking (BPR) is a classical pairwise objective for implicit-feedback recommendation [Rendle et al., 2009]. GRU4Rec models session histories with gated

recurrent units [Hidasi et al., 2015], while SASRec uses self-attention for next-item prediction [Kang and McAuley, 2018]. These methods rank items by scoring dense user and item representations.

**Generative recommendation.** Generative recommendation reframes next-item prediction as autoregressive item identifier generation. TIGER maps items into residual-quantized semantic IDs and trains a T5-style generator [Rajput et al., 2023]; ETEGRec studies end-to-end tokenizer–recommender training, motivating SEA-Rec’s alignment focus [Liu et al., 2025].

**Discrete representation learning.** Vector-quantized autoencoders learn discrete latent codes through reconstruction and commitment losses [van den Oord et al., 2017]. Residual quantization represents each item with multiple codebook entries; in SEA-Rec, these codes become compact generative item identifiers.

**Data mining perspective.** The project connects recommender modeling with data mining through sparsity analysis, long-tail characterization, item-code clustering, and codebook-utilization diagnostics. K-means initialization remains future work.

### 3 Problem Formulation

Let  $\mathcal{U}$  denote users and  $\mathcal{I}$  denote the item catalog. For user  $u \in \mathcal{U}$ , the historical sequence is

$$\mathbf{x}_u = (i_1, i_2, \dots, i_t), \quad i_j \in \mathcal{I}.$$

The goal is to predict the next item  $i_{t+1}$ . In full-catalog evaluation, the model ranks the target item against all items in  $\mathcal{I}$  rather than against sampled negatives.

For ranked list  $\pi_u$ , Recall@ $K$  is

$$\text{Recall@}K = \mathbb{I}[i_{t+1} \in \pi_u[:K]],$$

and NDCG@ $K$  is

$$\text{NDCG@}K = \begin{cases} 1/\log_2(r+1), & r \leq K, \\ 0, & r > K, \end{cases}$$

where  $r$  is the rank of the ground-truth item. We report metrics for  $K \in \{1, 5, 10\}$  and use NDCG@10 for validation.

## 4 Method

### 4.1 Architecture

SEA-Rec consists of an RQ-VAE item tokenizer  $\mathcal{M}_{\text{id}}$  and a T5-style encoder–decoder recommender  $\mathcal{M}_{\text{rec}}$ . The tokenizer maps each item embedding into discrete codes, and the recommender encodes historical item-code sequences to autoregressively decode the target code. Prefix-constrained beam search ensures generated code sequences correspond to valid catalog items.

### 4.2 Item Tokenizer

Each item  $i$  has a continuous embedding  $\mathbf{e}_i \in \mathbb{R}^{256}$ , implemented as a precomputed SASRec collaborative item embedding from the ETEGRec data. Thus, “semantic-enhanced” refers to structure in the collaborative embedding and discrete code space rather than raw natural-language metadata.

The RQ-VAE encoder maps  $\mathbf{e}_i$  to a latent vector, which is quantized by three learned residual codebook levels with  $K = 256$  codes per level:

$$\mathbf{c}_i = (c_i^1, c_i^2, c_i^3), \quad c_i^\ell \in \{0, \dots, K-1\}.$$

Each item has three learned residual quantization codes; the identifier is padded to length four only for compatibility with the original generative decoding format. The tokenizer objective is

$$\mathcal{L}_{\text{vq}} = \mathcal{L}_{\text{recon}}(\hat{\mathbf{e}}_i, \mathbf{e}_i) + \alpha \mathcal{L}_{\text{commit}},$$

where  $\hat{\mathbf{e}}_i$  is the reconstructed item embedding. This keeps discrete codes faithful to the original item features.

### 4.3 Generative Recommendation

The user history is flattened into a code sequence:

$$\mathbf{z}_u = [c_{i_1}^1, \dots, c_{i_1}^L, \dots, c_{i_t}^1, \dots, c_{i_t}^L],$$

where  $L$  is the generated code length. The recommender estimates

$$p(\mathbf{c}_{i_{t+1}} \mid \mathbf{x}_u) = \prod_{\ell=1}^L p(c_{i_{t+1}}^\ell \mid c_{i_{t+1}}^{<\ell}, \mathbf{x}_u).$$

The primary loss is code-level cross entropy:

$$\mathcal{L}_{\text{code}} = -\frac{1}{L} \sum_{\ell=1}^L \log p(c_{i_{t+1}}^\ell \mid c_{i_{t+1}}^{<\ell}, \mathbf{x}_u).$$

This objective trains the decoder to generate the target item code from the user’s history.

### 4.4 Alignment Objectives and Feature Adapter

SEA-Rec uses two auxiliary alignment losses. A symmetric KL loss aligns the code distributions induced by the encoded user sequence and target item:

$$\mathcal{L}_{\text{KL}} = \text{KL}(p_{\text{seq}} \parallel p_{\text{item}}) + \text{KL}(p_{\text{item}} \parallel p_{\text{seq}}).$$

A decoder-level contrastive loss aligns the decoder representation  $\hat{\mathbf{e}}_{\text{dec}}$  with the reconstructed target item representation  $\hat{\mathbf{e}}_{i^*}$ :

$$\mathcal{L}_{\text{dec-cl}} = \mathcal{L}_{\text{NCE}}(\hat{\mathbf{e}}_{\text{dec}}, \hat{\mathbf{e}}_{i^*}) + \mathcal{L}_{\text{NCE}}(\hat{\mathbf{e}}_{i^*}, \hat{\mathbf{e}}_{\text{dec}}).$$

A two-layer feature adapter maps the original 256-dimensional item features into the T5 hidden dimension and injects them into the encoder stream to compensate for quantization loss.

The full objective is

$$\mathcal{L} = w_{\text{code}} \mathcal{L}_{\text{code}} + w_{\text{vq}} \mathcal{L}_{\text{vq}} + w_{\text{KL}} \mathcal{L}_{\text{KL}} + w_{\text{dec-cl}} \mathcal{L}_{\text{dec-cl}}.$$

Training follows cyclic co-training: update  $\mathcal{M}_{\text{rec}}$  with the tokenizer frozen, update  $\mathcal{M}_{\text{id}}$  with the recommender frozen, refresh item codes, and finally fine-tune the recommender with the tokenizer fixed. Unlike ETEGRec’s end-to-end objective, SEA-Rec uses explicit KL alignment, decoder-level contrastive learning, and cyclic co-training to encourage item codes that are both informative and predictable.

## 5 Datasets and Experimental Setup

### 5.1 Dataset Analysis and Characterization

We analyze three Amazon Review subsets from the ETEGRec preprocessed data release: Game, Instrument, and Scientific. The datasets originate from Amazon Review 2023 and are 5-core filtered before leave-one-out splitting. Each record contains `user_id`, `inter_history`, and `target_id`; we verified that the processed splits contain no empty histories, missing targets, or duplicate validation/test users.

Table 1: Dataset statistics after 5-core filtering and leave-one-out preprocessing. Density is computed as training interactions divided by  $\# \text{users} \times \# \text{items}$ .

| Dataset    | #Users | #Items | Train int. | Density (%) | Avg. seq. | Avg. int./user | Avg. users/item |
|------------|--------|--------|------------|-------------|-----------|----------------|-----------------|
| Game       | 94,762 | 25,612 | 4,072,490  | 0.16780     | 7.68      | 42.98          | 160.63          |
| Instrument | 57,439 | 24,587 | 2,513,478  | 0.17798     | 7.40      | 43.76          | 102.70          |
| Scientific | 50,985 | 25,848 | 1,640,198  | 0.12446     | 6.31      | 32.17          | 64.22           |

Table 1 shows that all three datasets are extremely sparse: fewer than 0.18% of possible user–item pairs are observed in every case. Scientific is the most difficult subset by this criterion because it has the lowest interaction density, the shortest average sequence length, and the weakest user-side and item-side support.

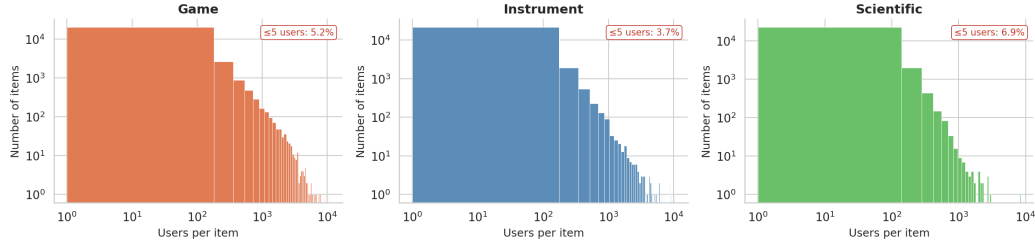


Figure 1: Item popularity distribution on a log-log scale. Most items appear in only a small number of user histories, producing a long-tail item distribution.

Figure 1 shows that item popularity is long-tailed in all three datasets: a small group of popular items receives many interactions, while many items have weak collaborative support. This makes purely independent item-ID modeling brittle, especially for rare items.

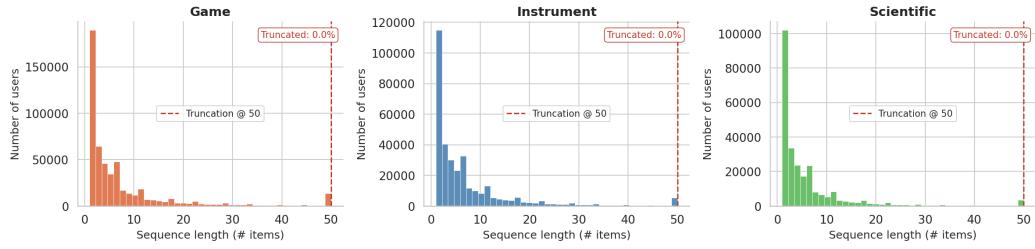


Figure 2: Distribution of user sequence lengths. The dashed line marks the 50-item truncation threshold used by the pipeline. No dataset effectively exceeds this limit after preprocessing.

Figure 2 shows that the main sequence-length problem is short context rather than truncation. The mean, median, and 95th-percentile training history lengths are (7.7, 4, 32) for Game, (7.4, 4, 27) for Instrument, and (6.3, 3, 23) for Scientific, all below the 50-item truncation threshold. Scientific is hardest because its median history length is only 3.

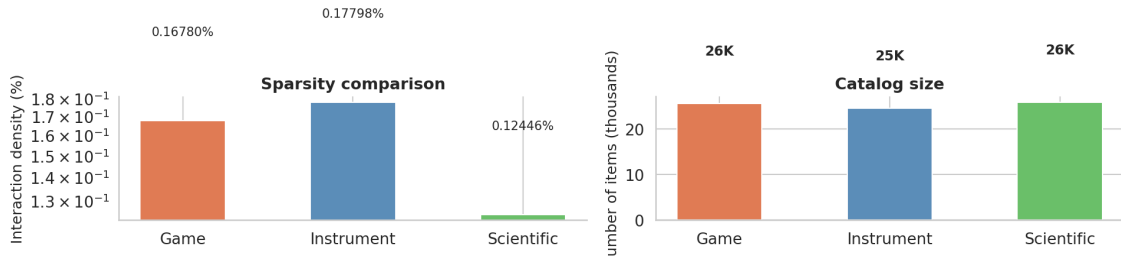


Figure 3: Cross-dataset comparison of interaction density and catalog size. The datasets have similar catalog sizes but different levels of interaction support.

Figure 3 separates catalog size from interaction density. Game and Scientific have similar catalog sizes, but Scientific is much sparser: 0.12446% density versus 0.16780% for Game. Instrument has the highest density at 0.17798%, so the datasets form a useful difficulty spectrum.

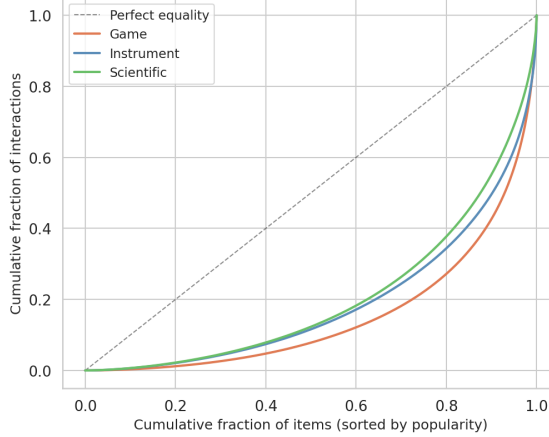


Figure 4: Lorenz curves of item popularity. Curves farther below the diagonal indicate stronger inequality in the distribution of item interactions.

Figure 4 summarizes item-popularity inequality using Lorenz curves. The Gini coefficients are 0.6916 for Game, 0.6202 for Instrument, and 0.5870 for Scientific, confirming that all three datasets remain long-tailed after 5-core filtering. Scientific does not have the highest Gini, but it is still the most difficult overall because it has the fewest users per item, the shortest histories, and the lowest density.

The random top-10 baseline contextualizes the metric scale. Under full-catalog ranking, expected random Recall@10 is  $10/|I|$ , or about  $4 \times 10^{-4}$  on these catalogs. SEA-Rec’s verified Scientific Recall@10 of 0.0419 is about 108 times larger, so low absolute values mainly reflect strict exact-match ranking over more than 25,000 candidate items.

Overall, these properties motivate SEA-Rec’s generative tokenizer–recommender design: residual-quantized codes let related items share structure in sparse, long-tailed catalogs where most user–item pairs are unobserved.

## 5.2 Preprocessing and Evaluation

The datasets use leave-one-out evaluation: for sequence  $[i_1, i_2, \dots, i_n]$ ,  $i_n$  is the test target,  $i_{n-1}$  is the validation target, and earlier interactions form training histories. Item features are precomputed 256-dimensional SASRec collaborative embeddings from ETEGRec, used by the RQ-VAE tokenizer and feature-adapter path.

All three SEA-Rec full-schedule runs are completed. Evaluation uses full-catalog ranking without sampled negatives; we report Recall@{1, 5, 10} and NDCG@{1, 5, 10}, with NDCG@10 for validation. Scientific is the main mechanism-analysis dataset and includes tuned BPR-MF, GRU4Rec, and SASRec baselines.

## 5.3 Implementation Details

The recommender uses 6 encoder/decoder layers, hidden size  $d = 128$ , feed-forward size  $d_{\text{ff}} = 512$ , 4 heads, and  $d_{kv} = 64$ . The RQ-VAE encoder is a 3-layer MLP,  $256 \rightarrow 512 \rightarrow 256 \rightarrow 128$ , with 3 residual quantization levels and  $K = 256$ . Training uses AdamW,  $lr_{\text{rec}} = 5 \times 10^{-3}$ ,  $lr_{\text{id}} = 10^{-4}$ , weight decay  $10^{-4}$ , cosine scheduling with 8000 warmup steps, batch size 64 per GPU on two GPUs, beam size 20, and maximum sequence length 210.

# 6 Results

## 6.1 Completed Full-Schedule Results

Table 2 reports completed full-schedule SEA-Rec runs. Game obtains the strongest absolute performance, Instrument is second, and Scientific is lowest, matching the dataset difficulty analysis. Final checkpoints improve over their corresponding pre-test checkpoints on every dataset, with the largest relative gain on Scientific.

Table 2: Completed full-schedule SEA-Rec results across all three datasets.  $\Delta$  values compare the final test checkpoint against the corresponding pre-test checkpoint.

| Dataset    | R@1    | R@5    | R@10   | N@5    | N@10   | $\Delta$ R@10 | $\Delta$ N@10 |
|------------|--------|--------|--------|--------|--------|---------------|---------------|
| Game       | 0.0167 | 0.0578 | 0.0908 | 0.0373 | 0.0479 | +19.0%        | +21.7%        |
| Instrument | 0.0118 | 0.0387 | 0.0603 | 0.0252 | 0.0322 | +9.6%         | +13.1%        |
| Scientific | 0.0075 | 0.0265 | 0.0419 | 0.0169 | 0.0219 | +28.8%        | +33.3%        |

Table 3: Comparison between SEA-Rec and ETEGRec.

| Dataset    | ETEGRec R@10 | SEA-Rec R@10  | ETEGRec N@10 | SEA-Rec N@10  |
|------------|--------------|---------------|--------------|---------------|
| Game       | 0.0389       | <b>0.0908</b> | 0.0244       | <b>0.0479</b> |
| Instrument | 0.0312       | <b>0.0603</b> | 0.0196       | <b>0.0322</b> |
| Scientific | 0.0341       | <b>0.0419</b> | 0.0211       | <b>0.0219</b> |

## 6.2 Comparison with the Original ETEGRec Framework

We compare SEA-Rec against ETEGRec under the same benchmark setting.

SEA-Rec improves Recall@10 across all three datasets, with the largest gains on Game and Instrument and a smaller positive improvement on Scientific.

## 6.3 Full-Catalog Baseline Comparison on Scientific

Table 4 compares SEA-Rec with BPR-MF, GRU4Rec, and SASRec on Scientific. SEA-Rec achieves the strongest metric values, improving over tuned BPR-MF by about  $3.2\times$  in Recall@10 and  $3.4\times$  in NDCG@10. The result supports residual-quantized generative retrieval with explicit tokenizer-recommender alignment in the sparsest dataset.

Table 4: Full-catalog comparison on Scientific. BPR-MF represents classical collaborative filtering, GRU4Rec and SASRec represent sequential recommendation, and all methods are evaluated by ranking against all catalog items.

| Method  | R@1           | R@5           | R@10          | N@5           | N@10          |
|---------|---------------|---------------|---------------|---------------|---------------|
| BPR-MF  | 0.0019        | 0.0075        | 0.0130        | 0.0046        | 0.0064        |
| GRU4Rec | 0.0026        | 0.0075        | 0.0133        | 0.0051        | 0.0069        |
| SASRec  | 0.0028        | 0.0083        | 0.0136        | 0.0055        | 0.0072        |
| SEA-Rec | <b>0.0075</b> | <b>0.0265</b> | <b>0.0419</b> | <b>0.0169</b> | <b>0.0219</b> |

## 6.4 Scientific Short-Run Ablations

Table 5 reports short-run Scientific ablations. These reduced-budget single-run comparisons are diagnostic rather than conclusive causal evidence.

Table 5: Scientific short-run ablations. All rows use the same reduced budget and are comparable only to one another.

| Variant             | KL  | Dec-CL | Adapter | R@1    | R@5    | R@10   | N@5    | N@10   | Best epoch |
|---------------------|-----|--------|---------|--------|--------|--------|--------|--------|------------|
| Full SEA-Rec short  | on  | on     | on      | 0.0074 | 0.0249 | 0.0399 | 0.0161 | 0.0210 | 13         |
| w/o KL loss         | off | on     | on      | 0.0072 | 0.0248 | 0.0399 | 0.0160 | 0.0209 | 13         |
| w/o Decoder-CL      | on  | off    | on      | 0.0072 | 0.0247 | 0.0398 | 0.0159 | 0.0208 | 13         |
| w/o Alignment       | off | off    | on      | 0.0071 | 0.0249 | 0.0402 | 0.0161 | 0.0210 | 13         |
| w/o Feature Adapter | on  | on     | off     | 0.0074 | 0.0256 | 0.0408 | 0.0165 | 0.0213 | 11         |

Removing KL, decoder contrastive learning, or both changes R@10 only slightly. Feature-adapter removal is also marginally higher, likely due to single-seed variance, limited budget, and already-strong SASRec item embeddings rather than evidence that the adapter is harmful.

## 6.5 Scope of Verified Comparisons

The verified conclusion is that SEA-Rec trains successfully across all three datasets, improves over corresponding pre-test checkpoints, outperforms verified BPR-MF, GRU4Rec, and SASRec baselines on Scientific, and maintains a non-collapsed codebook. Additional generative baselines such as TIGER remain future work, so we do not claim state-of-the-art performance.

## 7 Codebook Utilization and K-means Initialization Analysis

In generative recommendation, item identities are represented by discrete code sequences. If many codes are unused, the effective vocabulary shrinks and many items may share similar code paths. Codebook collapse would therefore reduce representation capacity and make decoding less informative. Table 6 summarizes the completed before/after codebook analysis on Scientific.

Table 6: Completed RQ-VAE codebook utilization analysis on Scientific.

| Level | Used codes | Util. | Dead | Entropy before → after |
|-------|------------|-------|------|------------------------|
| 0     | 256/256    | 100%  | 0    | 0.971 → 0.953          |
| 1     | 256/256    | 100%  | 0    | 0.997 → 0.970          |
| 2     | 256/256    | 100%  | 0    | 0.998 → 0.987          |

All three RQ-VAE levels use 256/256 codes after training, and dead codes remain zero. This indicates no codebook collapse. Entropy decreases slightly after training, suggesting mild recommendation-oriented specialization rather than collapse: code usage becomes somewhat less uniform, but the full vocabulary remains active. K-means initialization could initialize RQ-VAE codebooks with item-embedding cluster centroids, but the direct random-vs-k-means ablation was not completed and remains future work.

## 8 Limitations

This report is conservative about experimental claims. Full-schedule SEA-Rec runs are completed on Game, Instrument, and Scientific, but the complete ablation package and verified BPR-MF/GRU4Rec/SASRec comparisons are available only for Scientific. Generative baselines such as TIGER, multi-seed ablations, text embeddings, and random-vs-k-means codebook initialization remain future work. Short-run ablations should be read as directional diagnostics, and full-catalog exact-match evaluation may not reward semantically similar but non-identical recommendations.

## 9 Conclusion

SEA-Rec trains successfully across Game, Instrument, and Scientific, improves over corresponding pre-test checkpoints, outperforms verified BPR-MF, GRU4Rec, and SASRec baselines on Scientific, and maintains full RQ-VAE codebook utilization. The main data-mining lesson is that sparse interactions, short histories, and long-tailed popularity are central to the difficulty of strict full-catalog recommendation.

**Contributions and code.** Nguyen Tran Nhat Minh led implementation, full-schedule runs, ablations, logging, codebook analysis, and embedding checks; Trinh To Thao Nhi contributed tokenizer preparation, baselines, consistency checks, and result compilation; Nguyen Hoang Nam contributed framing, data analysis, report writing, and evaluation checks. Code and logs are available at [https://github.com/Mancupfire/Data\\_Mining](https://github.com/Mancupfire/Data_Mining).

## References

- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of UAI*, 2009.
- Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939*, 2015.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *Proceedings of ICDM*, 2018.
- Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, and Maheswaran Sathiamoorthy. Recommender systems with generative retrieval. In *Proceedings of NeurIPS*, 2023.
- Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. Generative recommender with end-to-end learnable item tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 729–739, 2025.
- Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Proceedings of NeurIPS*, 2017.